

ARAMIREZ-AI

ADR-018: Migración de API REST a GraphQL

Decisión arquitectónica · Comité de Arquitectura — Julio 2026

CONTEXTO

Contexto y Motivación

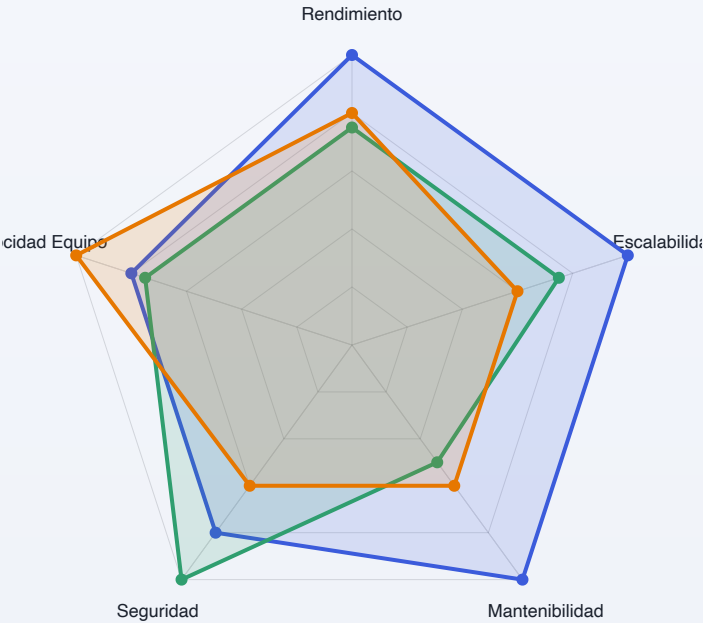
Problema Detectado

- **Overfetching en endpoints REST: el frontend recibe datos no utilizados en cada llamada**
- **Múltiples solicitudes HTTP para ensamblar una vista (problema N+1 en dashboards)**
- **Crecimiento descontrolado de endpoints REST: más de 85 endpoints en producción**
- **Tiempo de carga promedio de 2.8 segundos en el dashboard principal del cliente**
- **Dificultad para versionar la API sin romper contratos existentes con consumidores**

Opciones Evaluadas

Opción	Rendimiento	Mantenibilidad	Curva Aprendizaje	Coste	Votos
REST + JSON:API	3/5	4/5	2/5	\$0	2
GraphQL (Apollo Server)	5/5	5/5	4/5	\$500/mes	6
tRPC (TypeScript)	4/5	3/5	3/5	\$0	1
Mantener REST actual	2/5	2/5	1/5	\$0	0

Evaluación por Dimensión



Ventajas de GraphQL

- Un solo endpoint para toda la API
- Tipado fuerte con schema SDL auto-documentado
- Subscripciones en tiempo real nativas (WebSocket)
- Ecosistema Apollo Server maduro y probado
- Herramientas como GraphiQL para explorar la API
- Elimina el sobre-fetching y sub-fetching

Riesgos Identificados

- Complejidad en caché a nivel HTTP (no aplica como REST)
- Problema N+1 en resolvers requiere DataLoader
- Límite de peticiones más complejo de implementar
- Migración gradual requiere coexistencia REST+GraphQL
- Curva de aprendizaje para el equipo de backend

GraphQL + Apollo

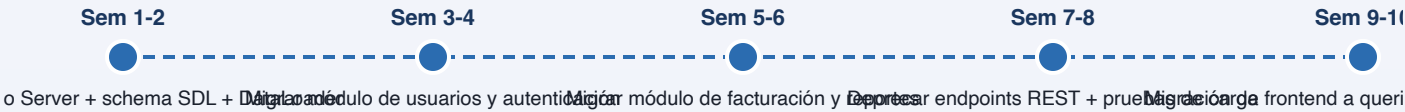
Decisión Final

Aprobado por unanimidad - Implementación en fases durante Q3 2026

“ GraphQL nos permite evolucionar la API sin versionado explícito. El frontend pide exactamente lo que necesita y nada más. Es la herramienta adecuada para un ecosistema de múltiples consumidores.

— Líder Técnico de Plataforma

Plan de Migración por Fases



Flujo de Decisión del ADR

ARAMIREZ-AI

